

AWS GenAI 課程 · 第 5 單元

# 帳號安全、IAM 與成本控制

盧建霖 • 2026-09-19

一組金鑰貼錯地方，對方就是你。  
今天把帳號做到沒有金鑰可以漏。

# Agenda

- 01 選區：Region、AZ 與邊緣
- 02 身分與權限：Policy 與 Role
- 03 六大支柱與成本控制
- 04 Agent 的權限、LLM 的錢與實作

## TODAY'S GOAL

拿到一個空帳號，能照鎖 root、開身分、接工具、拉保險絲四步先把自己的身分開好。

---

前三段講觀念，最後一段套到 agent 上並動手做；實作分必做、進階、加分三層，只交必做就及格。

# 先定義名詞，後面才好講

這六個詞今天會反覆出現，第一次出現就先給名字

## 區域 **Region**

一個地理位置，例如東京，裡面裝著多個可用區域。

## 可用區域 **AZ**

一個或多個各自獨立供電供網的資料中心。

## 政策 **Policy**

一份 JSON 文件，寫誰能對什麼做什麼；權限只寫在這裡。

## 角色 **Role**

沒有密碼沒有金鑰的身分，誰來扮演誰就暫時拿到它的權限。

## 存取金鑰 **access key**

一組 AKIA 開頭的 ID 加 secret，等同你的帳號密碼。

## 預算動作 **Budget Action**

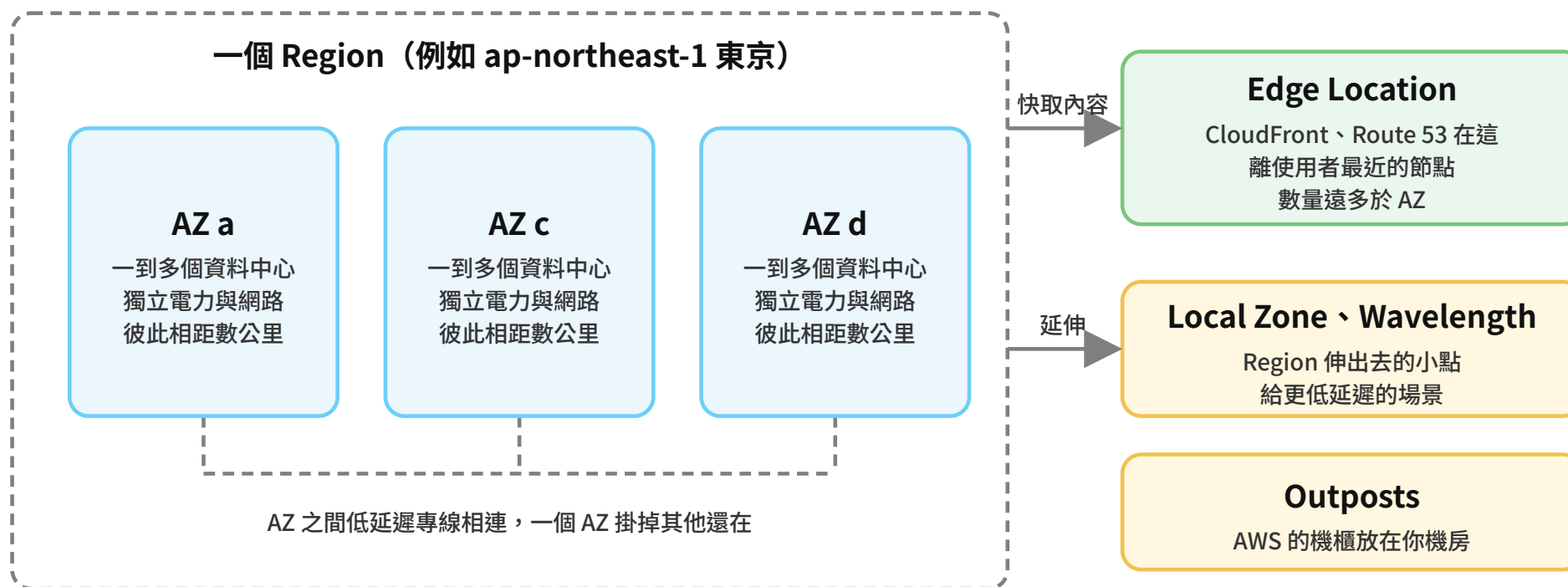
預算達門檻時自動執行的動作，例如套一份 Deny 的政策。

# 右上角的下拉選單，選哪一區

- 一、**Region** 裝 **AZ**，**Edge** 在外面而且數量最多
- 二、選區順序：法規、延遲、服務、價格
- 三、多 **AZ** 擋機房出事，全球部署選多 **Region**

# Region 裝 AZ，Edge 在外面

Edge Location（邊緣節點）是離使用者最近的快取點，不屬於任何 AZ



- 1 每個新 Region 至少三個 AZ，AZ 之間有低延遲專線相連
- 2 Edge Location 在 Region 外面，數量遠多於 Region
- 3 Local Zone、Wavelength、Outposts 是 Region 往外伸的三種延伸

# 三種延伸，差在機器放在誰家

三者都掛在一個母 Region 底下，在同一個 VPC 裡開子網路、開 EC2

| 延伸         | 機器放在哪       | 誰的機房         | 解決什麼           |
|------------|-------------|--------------|----------------|
| Local Zone | 人口與產業集中的大城市 | AWS          | 那座城市的使用者要低延遲   |
| Wavelength | 電信商網路的邊緣    | 電信商          | 手機等行動裝置要低延遲    |
| Outposts   | 你公司的機房      | 你的，硬體是 AWS 的 | 資料留在公司、同一套 API |

Edge Location 只做快取與 DNS，不能開你的 EC2；這三個都能開。

# 考題看訊號詞：地端就是 Outposts

題庫考這一塊，幾乎都在考 Outposts

## 1 地端、內部部署、同一套 API → Outposts

資料必須留在本地又要跟 AWS 低延遲、部分工作負載留地端，答案都是它。

---

## 2 行動裝置、電信網路 → Wavelength

應用程式要貼近手機使用者。

---

## 3 某個大城市、即時遊戲或直播 → Local Zone

全球快取靜態內容則是 Edge Location，不是這三個。

判斷關鍵：資料還在不在 AWS 的地盤上。只有 Outposts 真的在你家。

# 一個 AZ 就是一棟資料中心？

## × 大家通常會想

AZ 就是一棟資料中心。

這樣想會漏掉它的重點：電力跟網路。

## ✓ 實際上

一個 AZ 可以是好幾棟，重點是它跟隔壁 AZ 不共用電力跟網路。

一個 AZ 停電另一個還活著，所以跨 AZ 部署是高可用的第一步。

# 選區問四件事，法規排第一

| 問題         | 為什麼重要          | 例子                      |
|------------|----------------|-------------------------|
| 法規允許資料放哪   | 有些行業規定資料不能離開國境 | 金融個資留台灣選 ap-east-2 台北   |
| 使用者在哪      | 距離就是延遲         | 使用者在台灣，東京比維吉尼亞快         |
| 那一區有沒有這個服務 | 新服務先開美東，其他區慢慢補 | Bedrock 某些模型只在特定 Region |
| 價格         | 同一台機器各區價格不同    | 美東通常最便宜                 |

順序是法規、延遲、服務、價格；法規是硬的，其他是軟的。

# 多 AZ 擋機房，多 Region 擋城市

大小關係：Edge Location 最多，AZ 其次，Region 最少；精確數字每年在變，不用背

## 1 多可用區：同一個 Region 裡跨 AZ

擋的是一個資料中心出事。

---

## 2 多區域：跨地理位置

擋的是整個城市出事，或想服務全球使用者；全球部署選多區域。

---

## 3 只有從網際網路傳進 AWS 免費

Region 之間的傳輸要收費，同一 Region 內 AZ 之間也有少量費用。

主控台一次只顯示一個 Region 的資源；找不到機器，先切右上角的下拉選單。

QUIZ 01 · CLF-C02

一家公司希望使用 **AWS Cloud** 在全球範圍內部署應用程式。該公司應該使用哪一種架構部署模型來滿足此需求？

**A** 多區域 (Multi-Region)

**B** 單一區域 (Single-Region)

**C** 多可用區 (Multi-AZ)

**D** 單一可用區 (Single-AZ)

想 30 秒，等一下翻下一頁對答案。

QUIZ 01 · CLF-C02 · ANSWER

一家公司希望使用 **AWS Cloud** 在全球範圍內部署應用程式。該公司應該使用哪一種架構部署模型來滿足此需求？

**A** 多區域 (Multi-Region)

**B** 單一區域 (Single-Region)

**C** 多可用區 (Multi-AZ)

**D** 單一可用區 (Single-AZ)

A。「全球部署」要的是多個 Region；Multi-AZ 還在同一個地理區域裡，解的是機房掛掉，不是離全球使用者近。

# 動手做：看到東京有幾個 **AZ**，台北 **Region** 預設沒開 **(1/2)**

| 小步 | 做什麼                                                                                                                                                                       | 做完應該看到                                                   |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| 1  | 登入主控台，在上方導覽列點目前顯示的 Region 名稱                                                                                                                                              | 展開一串 Region 清單，<br>每一列是一個地理位置                            |
| 2  | 在清單裡選 Asia Pacific (Tokyo)                                                                                                                                                | 導覽列換成東京，網址<br>列出現 region=ap-northeast-1                  |
| 3  | 另開分頁到<br><a href="https://console.aws.amazon.com/awsglobalview/home#RegionsAndZones">https://console.aws.amazon.com/awsglobalview/home#RegionsAndZones</a> ，停在 Regions 分頁 | 表格有 Status 欄，Asia Pacific (Tokyo) 那列是 Enabled by default |

# 動手做：看到東京有幾個 AZ，台北 Region 預設沒開（2/2）

| 小步 | 做什麼                                     | 做完應該看到                                                           |
|----|-----------------------------------------|------------------------------------------------------------------|
| 4  | 在同一張表找 Asia Pacific (Taipei)            | Status 是 Disabled（除非有人開過）；2019 年 3 月 20 日之後推出的 Region 都要先手動開啟才能用 |
| 5  | 勾選 Tokyo 那一列，按 View details，切到 Zones 分頁 | Zone type 是 Availability zone 的列至少三列，Zone ID 欄是 apne1-az 開頭加數字   |

這一步讓你親眼看到本課「Region 裝 AZ」那一段講的事，也看到台北這區離你最近但要先開啟才能用。注意這張表只告訴你 Region 開了沒，沒告訴你某個服務在那一區有沒有，那要去 Regional Services 頁查，個人作業會做。

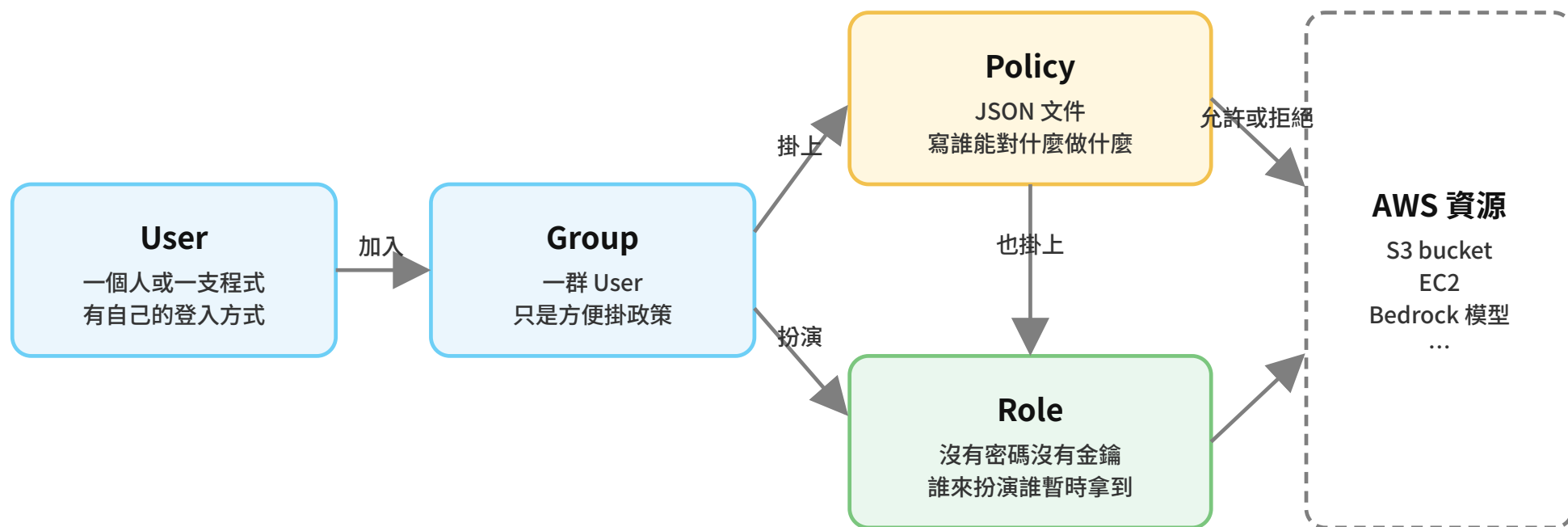
# 權限只寫在 **Policy**，金鑰最好不存在

- 一、**User**、**Group**、**Role** 本身沒有權限，**Policy** 才有
- 二、沒寫到就是拒絕；**Allow** 跟 **Deny** 撞到，**Deny** 贏
- 三、人走 **Identity Center**，程式走 **Role**，都拿臨時憑證

# 權限只在 **Policy**，其他三個是掛的地方

IAM (Identity and Access Management)：AWS 的身分與權限服務，決定誰能對哪個資源做什麼

權限永遠寫在 Policy 裡，User、Group、Role 只是 Policy 掛的地方



- 1 黃框 Policy 是唯一寫著權限的東西
- 2 用 Group 指派權限：十個人改權限只改一次，不要一人一份
- 3 綠框 Role 沒有密碼沒有金鑰，誰來扮演誰暫時拿到

# 五個身分名詞，日常不用 root

MFA 是 Multi-Factor Authentication (多重要素驗證)；root 不能被 policy 限制

| 名詞        | 白話               | 什麼時候用                  |
|-----------|------------------|------------------------|
| root user | 建帳號那個 email，權限無限 | 只開 MFA、改帳單資訊，日常不登入     |
| User      | 一個人或一支程式的長期身分    | 有密碼或 access key，現在盡量不用 |
| Group     | 一群 User          | 把 Policy 一次掛給很多人       |
| Policy    | JSON 文件          | 一切權限的來源                |
| Role      | 沒有密碼沒有金鑰的身分      | 誰扮演誰暫時拿到它的 Policy      |

# 一條 Statement 就是一個句子

允許或拒絕、做什麼動作、對哪個資源、在什麼條件下

一條 Statement 就是一個句子：誰（掛在誰身上）+ Effect + Action + Resource + Condition

## Effect

Allow 或 Deny  
Deny 永遠贏

## Action

s3:GetObject  
動詞，服務名加冒號

## Resource

arn:aws:s3:::my-bucket/\*  
受詞，ARN 格式

## Condition

aws:MultiFactorAuthPresent  
什麼情況下才算數

## 沒寫到的一律拒絕（default deny）

Allow 只開你寫的那幾扇門；同時有 Allow 跟 Deny 時，Deny 贏  
最小權限就是：Action 寫具體動詞、Resource 寫具體 ARN、不要用 \*

- 1 Effect、Action、Resource 必填，Condition 選填
- 2 Resource 填 ARN（Amazon Resource Name），AWS 資源的完整地址
- 3 兩條規則：沒寫到就是拒絕；Allow 跟 Deny 撞到，Deny 贏

# 把 **policy** 唸成一句中文

有 MFA 的情況下，允許列出 course-data 這個 bucket 跟讀裡面的檔案

```
policy.json

{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:ListBucket"],
    "Resource": [
      "arn:aws:s3:::course-data",
      "arn:aws:s3:::course-data/*"
    ],
    "Condition": {"Bool": {"aws:MultiFactorAuthPresent": "true"}}
  ]
}
```

1 沒寫到的動作，例如 `s3:PutObject`，一律不行

2 一條 `Deny` 可以把危險動作硬鎖掉，例如禁止刪 bucket

# 最小權限：先跑、看記錄、再收緊

| 做法                  | 白話                              |
|---------------------|---------------------------------|
| 人走聯合登入              | 人不要有 IAM User，登入拿臨時憑證           |
| 程式走 Role            | EC2、Lambda 用 Role，不塞 access key |
| 全員 MFA              | root 跟每個人都開                     |
| IAM Access Analyzer | 看誰實際用了什麼，產出剛剛好的 policy          |
| credential report   | 一份 CSV：每個 User 的密碼、金鑰、MFA 狀態    |

agent 拿的權限有多寬，它做錯事的範圍就有多寬。

## QUIZ 02 · CLF-C02

使用 **AWS Identity and Access Management (IAM)** 僅授予完成任務所需的資源存取權，這個概念被稱為：

**A** 受限存取 (restricted access)

**B** 按需存取 (as-needed access)

**C** 最小權限存取 (least privilege access)

**D** 權杖存取 (token access)

想 30 秒，等一下翻下一頁對答案。

使用 **AWS Identity and Access Management (IAM)** 僅授予完成任務所需的資源存取權，這個概念被稱為：

**A** 受限存取 (restricted access)

**B** 按需存取 (as-needed access)

**C** 最小權限存取 (least privilege access)

**D** 權杖存取 (token access)

C。「僅授予完成任務所需」就是最小權限的定義句；A、B 不是 AWS 的正式名詞，D 講的是憑證的形式不是範圍。

# 動手做：把 AmazonS3ReadOnlyAccess 唸成一句中文 (1/2)

| 小步 | 做什麼                                                                                                               | 做完應該看到                                 |
|----|-------------------------------------------------------------------------------------------------------------------|----------------------------------------|
| 1  | 開 IAM 主控台 <a href="https://console.aws.amazon.com/iam/">https://console.aws.amazon.com/iam/</a> ，左側導覽窗格點 Policies | 一長串 policy 清單，上方有搜尋框                   |
| 2  | 搜尋框輸入 AmazonS3ReadOnlyAccess                                                                                      | 清單只剩名稱相符的那一列                           |
| 3  | 點 policy 名稱，停在 Permissions 分頁                                                                                     | 一張 policy 摘要表，列出這份 policy 碰到的服務，S3 在裡面 |

# 動手做：把 AmazonS3ReadOnlyAccess 唸成一句中文 (2/2)

| 小步 | 做什麼                                                | 做完應該看到                                                                                                                           |
|----|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| 4  | 在 Permissions 分頁把 Summary 切成 JSON                  | 看到一條 Statement：Effect 是 Allow；Action 有 s3:Get*、s3:List*、s3:Describe*、s3-object-lambda:Get*、s3-object-lambda:List*；Resource 是 "*" |
| 5  | 不看教材，用本課的句型把它唸成一句中文，再找找看 s3:PutObject 在不在 Action 裡 | 你的句子接近「允許對所有資源做 S3 開頭是 Get、List、Describe 的動作」；PutObject 不在清單，這份 policy 沒有授權上傳；身上只掛這一份、bucket 也沒另外放行的身分去上傳會被拒絕                    |

這一步讓你親眼看到本課「一條 Statement 就是一個句子」跟「沒寫到就是拒絕」那兩段講的事；Resource 寫 "\*"，官方提醒過 AWS managed policy 是給所有客戶用的，不一定符合你的最小權限，拿來起步可以，之後要收緊。

# 長期金鑰外洩，要靠人去刪

access key 建好就永久有效；Role 那條路換到的憑證會過期



- 1 左邊紅色：外洩後對方就是你，直到有人發現、手動刪掉
- 2 右邊綠色：先證明身分，再換一組會過期的憑證
- 3 兩邊差在出事之後要不要靠人去刪

# 金鑰通常這樣漏出去

| 外洩的典型路徑             | 為什麼常發生                                          |
|---------------------|-------------------------------------------------|
| 推進 <b>GitHub</b>    | 忘了 <code>.gitignore</code> ，公開 repo 幾分鐘內就被機器人掃到 |
| 貼給 <b>AI</b>        | 把整份設定檔貼進對話請它除錯                                  |
| 寫死在程式裡              | <code>hard code</code> 在應用程式裡，題庫的反面選項           |
| 筆電或 <b>CI</b> 環境被拿走 | <code>credentials</code> 檔就在那裡                  |

環境建置時開的那組金鑰是過渡，今天的實作會換掉。

# Role 是一件外套，穿上才拿到憑證

STS (Security Token Service) 是 AWS 發臨時憑證的服務，AssumeRole 就是跟它要一組

## 1 先證明身分，信任政策說誰能扮演

信任政策 (trust policy) 是 Role 上的第二份 policy，寫誰能穿這件外套。

---

## 2 STS AssumeRole 換到臨時憑證

key 加 secret 加 session token；AWS 用第三個值認出這是暫時的。

---

## 3 預設 1 小時，最長 12 小時，到期自動作廢

不用任何人去刪。

一小時到了程式不會斷：SDK 跟 CLI 會在到期前自動再要一組。

# 人走 Identity Center，程式走 Role

IAM Identity Center：AWS 的單一登入服務，一組帳號密碼加 MFA 進所有 AWS 帳號

| 誰                | 怎麼拿身分                                                          |
|------------------|----------------------------------------------------------------|
| 人                | Identity Center 登入，選一組權限（permission set）                       |
| 人用 CLI           | <code>aws configure sso</code> 建好，再 <code>aws sso login</code> |
| EC2 上的程式         | 開機器時掛一個 Role                                                   |
| 另一個 AWS 帳號的人     | 對方 Role 的信任政策列你的帳號，你 AssumeRole                                |
| 公司已有 Google 或 AD | 接進 Identity Center 當身分來源，不用第二組密碼                               |

每條路拿到的都是臨時憑證；IAM User 加 access key 只留給真的沒辦法的工具。

### QUIZ 03 · CLF-C02

一家公司希望授予某個 **AWS** 帳戶中的使用者存取另一個 **AWS** 帳戶中的資源的權限。這些使用者目前尚未被授予存取這些資源的權限。以下哪一項 **AWS** 服務可以滿足此需求？

A

IAM group

B

IAM role

C

IAM tag

D

IAM Access Analyzer

想 30 秒，等一下翻下一頁對答案。

### QUIZ 03 · CLF-C02 · ANSWER

一家公司希望授予某個 **AWS** 帳戶中的使用者存取另一個 **AWS** 帳戶中的資源的權限。這些使用者目前尚未被授予存取這些資源的權限。以下哪一項 **AWS** 服務可以滿足此需求？

**A** IAM group

**B** IAM role

**C** IAM tag

**D** IAM Access Analyzer

B。「一個帳戶的使用者要用另一個帳戶的資源」是跨帳號存取，靠對方帳戶建一個 Role 讓你 AssumeRole；Group 出不了帳戶，tag 是標籤，Access Analyzer 是檢查工具。

## 動手做：看一個 **Role** 身上的兩份 **policy**，再找它的金鑰 (1/2)

| 小步 | 做什麼                                                                                                            | 做完應該看到                                        |
|----|----------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| 1  | 用 root 登入的人：右上角帳號名稱 → Security credentials，捲到 Access keys 那一區（用其他身分登入的人跳過這步，看講師投影 root 的畫面）                    | 列出 root 名下的 access key；理想是一把都沒有               |
| 2  | 開 IAM 主控台 <a href="https://console.aws.amazon.com/iam/">https://console.aws.amazon.com/iam/</a> ，左側導覽窗格點 Roles | 角色清單裡有 AWSServiceRoleForSupport               |
| 3  | 點 AWSServiceRoleForSupport 的名稱進它的詳細頁；上半部是 Summary，下面有一排分頁，先停在 Permissions 分頁                                   | 掛著一份 AWSSupportServiceRolePolicy，這是「穿上外套能做什麼」 |

## 動手做：看一個 **Role** 身上的兩份 **policy**，再找它的金鑰 (2/2)

| 小步 | 做什麼                                    | 做完應該看到                                                                                      |
|----|----------------------------------------|---------------------------------------------------------------------------------------------|
| 4  | 同一頁、同一排分頁，點 <b>Trust relationships</b> | 一份 JSON，Principal 裡是 "Service": "support.amazonaws.com"，Action 是 sts:AssumeRole，這是「誰能穿這件外套」 |
| 5  | 在這個 Role 的頁面上找 <b>access key</b> 或密碼   | 找不到。Role 本來就沒有長期憑證，誰扮演它，誰才拿到一組會過期的臨時憑證                                                      |

這一步讓你親眼看到本課「Role 是一件外套」那一段講的兩份 policy，以及「Role 沒有長期金鑰」那句話的實物。

# 六個角度替設計辯護，四個工具管住帳單

- 一、每個決策問六次，知道自己放棄了哪一根柱子
- 二、**AWS** 管雲本身，你管雲裡面，分界線隨服務移動
- 三、估、守、看、歸屬；只有 **Budget Action** 會動手

# 六根柱子，每根一組設計原則

Well-Architected Framework：AWS 整理的架構設計原則，分六個面向

AWS Well-Architected Framework 六大支柱，每根支柱底下是一組設計原則



- 1 每格上面中文名、中間英文名、下面一句代表性設計原則
- 2 黃色那格安全性是這個單元的主題
- 3 六個不用都拿滿分；每個決策問六次，知道放棄了哪一個

# 每根柱子問一個問題

記柱子的問題，原則屬於哪一根就不會混

## 營運卓越

你怎麼跑、怎麼改、怎麼從失敗學。

## 安全性

誰能碰什麼、追得到嗎。

## 可靠性

壞了會自己好嗎。

## 效能效率

用對資源了嗎。

## 成本最佳化

錢花在刀口上嗎。

## 永續性

用得夠滿嗎。

## 三句常混的原則，看它在問什麼

| 原則           | 屬於     | 記法              |
|--------------|--------|-----------------|
| 小步、頻繁、可逆的變更  | 營運卓越   | 在問怎麼改           |
| 自動從失敗復原、水平擴展 | 可靠性    | 在問壞了會不會自己好      |
| 用資料驅動架構      | 通用設計原則 | 六根柱子共用，共通點是別用猜的 |

通用原則五句：不猜容量、在正式規模測試、自動化、可演化、用資料跟演練改進。

# 評估上雲準備度，用 **Well-Architected** ？

## × 大家通常會想

公司要評估準備好上雲了沒，拿 **Well-Architected Framework** 來評。

兩個框架名字長得像，題庫就考這個區分。

## ✓ 實際上

遷移前的評估用 **CAF (Cloud Adoption Framework)**；**Well-Architected** 評的是架構本身。

Well-Architected Tool 是主控台裡照六支柱問你問題的自評工具。

# AWS 管雲本身，你管雲裡面

共同責任模型（shared responsibility model）：AWS 管雲本身，你管放進雲裡的東西

| 誰   | 負責什麼            | 例子                              |
|-----|-----------------|---------------------------------|
| AWS | 實體機房、硬體、網路、虛擬化層 | 資料中心門禁、RDS 的自動備份機制              |
| 你   | 資料、身分與權限、作業系統補丁 | IAM policy、security group、S3 加密 |
| 分界線 | 隨服務移動           | EC2 管到作業系統，Lambda 只管程式碼         |

口訣：這件事 AWS 碰不碰得到？資料內容、權限設定、加密選項，AWS 不碰就是你的。

QUIZ 04 · CLF-C02

**AWS Well-Architected Framework** 的哪一個支柱與能夠對 **AWS** 雲端架構進行頻繁、小型且可逆的變更相符？

A

安全性 (Security)

B

成本優化 (Cost optimization)

C

營運卓越 (Operational excellence)

D

效能效率 (Performance efficiency)

想 30 秒，等一下翻下一頁對答案。

**AWS Well-Architected Framework** 的哪一個支柱與能夠對 **AWS** 雲端架構進行頻繁、小型且可逆的變更相符？

**A** 安全性 (Security)

**B** 成本優化 (Cost optimization)

**C** 營運卓越 (Operational excellence)

**D** 效能效率 (Performance efficiency)

**C**。「頻繁、小型、可逆的變更」是營運卓越支柱的設計原則原句；安全講保護，成本講省錢，效能講選對資源。

## 動手做：用 Well-Architected Tool 審第一題就刪掉 (1/2)

| 小步 | 做什麼                                                                                                                                                                                                | 做完應該看到                                                                                                       |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| 1  | Region 切到 Asia Pacific (Tokyo)，開 <a href="https://console.aws.amazon.com/wellarchitected/">https://console.aws.amazon.com/wellarchitected/</a> ，點 Define workload（或左側 Workloads → Define workload） | 一張表單，有 Name、Description、Review owner、Environment、Regions 等欄位                                                 |
| 2  | Name 填 course-demo-wa，Description 填「課堂示範」，Review owner 填講師 email，Environment 選 Pre-production，Regions 勾 AWS Regions 裡的 Asia Pacific (Tokyo)，按 Next                                                 | 進到 Apply Profile 這一步，選填                                                                                      |
| 3  | 不選 profile，按 Next；在選 lens 那一頁不動，按 Define workload                                                                                                                                                  | 回到這個 workload 的詳細頁；AWS Well-Architected Framework lens 已自動套上（官方 Lenses 頁寫明它在定義 workload 時自動套用，2026-09-24 查閱） |

## 動手做：用 Well-Architected Tool 審第一題就刪掉 (2/2)

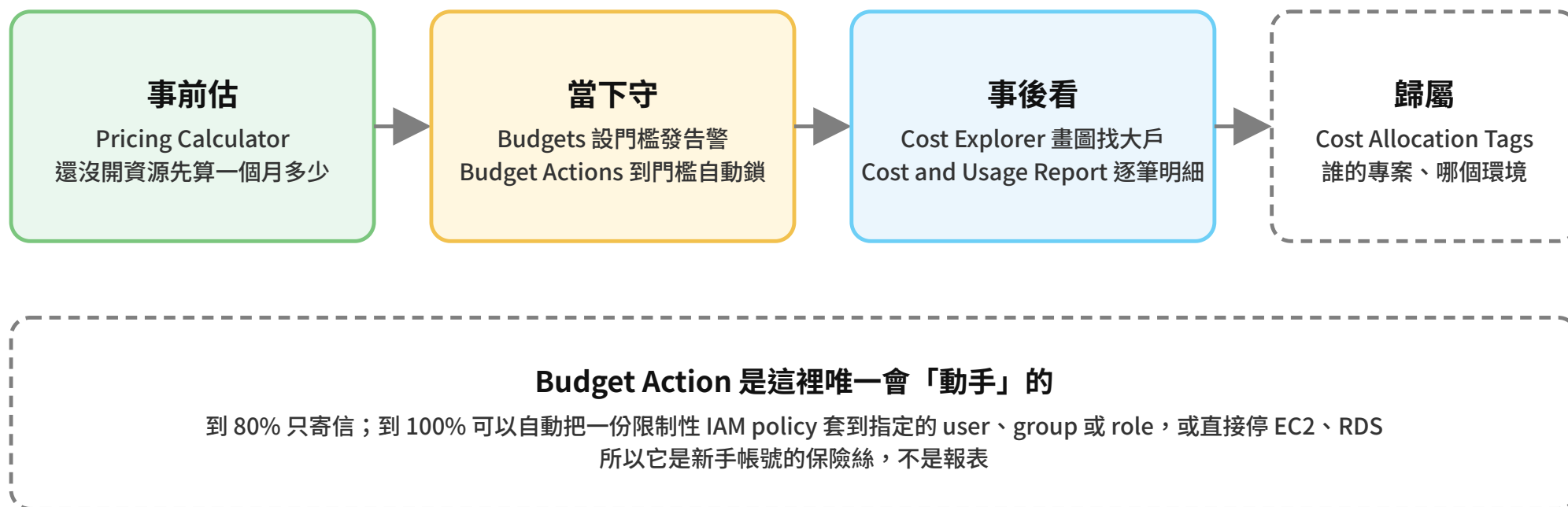
| 小步 | 做什麼                                                                  | 做完應該看到                                                                                                                             |
|----|----------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| 4  | 按 Start reviewing                                                    | 出現第一題（講師 2026-09 示範時是營運卓越的「OPS 1. How do you determine what your priorities are?」，題目文字會隨版本改），下面是一串 best practice 可以勾，右邊 Info 可以看說明 |
| 5  | 不作答，按 Save and exit                                                  | 回到 workload 詳細頁，按鈕變成 Continue reviewing，下次從這裡接著審                                                                                   |
| 6  | 清理：左側 Workloads → 勾 course-demo-wa → Delete → 在 Delete 視窗再按一次 Delete | Workloads 清單裡沒有 course-demo-wa                                                                                                     |

這一步讓你親眼看到本課「拿六根柱子檢視每個決策」那一段講的事：工具照六根柱子一題一題問你做了哪些最佳實務。

# 四個工具照時間排：估、守、看、歸屬

花之前、花的當下、花完之後，再加一個說清楚誰花的

錢花掉之前、花的當下、花完之後，各有一個工具；第四個負責說清楚是誰花的



- 1 黃色那格 **Budgets** 是唯一會在花的當下攔你的
- 2 四個各管一段，沒有一個能取代另一個
- 3 下排虛線框：**Budget Action** 是新手帳號的保險絲

# 每個工具回答一個問題

CUR 是 Cost and Usage Report，最細的逐筆帳單，倒進 S3

| 工具                   | 管哪一段 | 回答什麼問題         |
|----------------------|------|----------------|
| Pricing Calculator   | 事前   | 這個架構一個月大概多少    |
| Budgets              | 當下   | 這個月會不會爆        |
| Cost Explorer        | 事後   | 錢花在哪個服務、哪台機器太大 |
| CUR                  | 事後   | 財務要逐筆對帳、要自動產報表 |
| Cost Allocation Tags | 歸屬   | 這筆是哪個專案、哪個環境   |

# Budget Action：寄信升級成自動拉閘

月成本預算設你能接受的數字：80% 通知，100% 動作

## 1 實際或預測到 80%：通知

80% 還來得及查；預測型告警月中就告訴你照這速度月底會爆。

---

## 2 到 100%：執行 Budget Action

套一份限制性 IAM policy 到指定 user、group、role，或直接停 EC2、RDS。

---

## 3 Action 的 policy 用 Deny

例如 Deny `ec2:RunInstances`，靠的是「Deny 贏」那條規則。

再建一筆零花費預算，門檻 0.01 美元；Action 可以設成人工核准才執行。

# 事後看帳：什麼情境用什麼

| 情境                            | 用什麼                                                |
|-------------------------------|----------------------------------------------------|
| 這個月多 <b>200</b> 美元，哪來的        | <b>Cost Explorer</b> 按服務分組                         |
| <b>EC2</b> 一直只用 <b>5% CPU</b> | <b>Cost Explorer</b> 的建議或 <b>Compute Optimizer</b> |
| 花費突然跟平常不一樣                    | <b>Cost Anomaly Detection</b> ，開了自動寄信              |
| 三個專題共用帳號，各花多少                 | 每個資源貼 <b>Project</b> 標籤並啟用                         |
| 三個團隊各要一份帳單                    | <b>Organizations</b> ，一個團隊一個帳號                     |

多個團隊分帳靠**多帳號**；同一帳號內分攤才靠標籤。

## 貼了標籤，帳單就看得得到？

### × 大家通常會想

資源貼上 **Project** 標籤，帳單就會按專案分好。

貼了沒啟用，帳單看不到。

### ✓ 實際上

要到 **Billing** 啟用成 **cost allocation tag** 才算數。

啟用後最多 24 小時才出現在報表。

## QUIZ 05 · CLF-C02

哪一項 **AWS** 服務或工具可協助使用者將支出與使用情況隨時間進行視覺化、理解並加以管理？

A

AWS Organizations

B

AWS Pricing Calculator

C

AWS Cost Explorer

D

AWS Service Catalog

想 30 秒，等一下翻下一頁對答案。

QUIZ 05 · CLF-C02 · ANSWER

哪一項 **AWS** 服務或工具可協助使用者將支出與使用情況隨時間進行視覺化、理解並加以管理？

A AWS Organizations

B AWS Pricing Calculator

C AWS Cost Explorer

D AWS Service Catalog

C。「把支出與使用量隨時間視覺化」是 Cost Explorer，教材的「事後看」；Pricing Calculator 是還沒花錢的估算。

# 動手做：估一台東京的小機器一個月多少錢 (1/2)

| 小步 | 做什麼                                                         | 做完應該看到                                |
|----|-------------------------------------------------------------|---------------------------------------|
| 1  | 開 <code>https://calculator.aws/#/</code> ，按 Create estimate | 進到 Add service 頁，一整排可以估價的服務，上方有搜尋框    |
| 2  | 搜尋框輸入 Amazon EC2，在那一格按 Configure                            | 進到 EC2 的設定頁，有 Description 跟 Region 欄位 |
| 3  | Description 填 course-demo，Region 選 Asia Pacific (Tokyo)     | Region 欄顯示 Asia Pacific (Tokyo)       |

## 動手做：估一台東京的小機器一個月多少錢 (2/2)

| 小步 | 做什麼                                                                                                | 做完應該看到                                                           |
|----|----------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| 4  | 在 EC2 specifications 區確認 Tenancy 是 Shared Instances、作業系統是 Linux、台數是 1（三個都是預設值），再用搜尋列找 t3.micro 並選它 | 機型那一欄選定 t3.micro                                                 |
| 5  | Payment options 的計價方式選 On-Demand，預期使用率填 100%（整個月開著）；往下到 Amazon EBS 區把儲存量填 0                        | 頁面顯示 upfront 跟 monthly 兩個金額；EBS 填 0，所以這是「只算運算」的月費，真的開機器還要加開機磁碟的錢 |
| 6  | 按 Save and add service，再按 View summary                                                             | My estimate 頁顯示預付、每月、12 個月三種金額，下面列著剛加的 EC2                       |

這一步讓你親眼看到本課「估用 Pricing Calculator」那一段講的事，錢還沒花就知道一個月大概多少，之後設 Budgets 門檻才有依據。

# Agent 是員工，給它自己的識別證跟預算

- 一、**agent** 掛專用 **Role**，不借你的身分
- 二、**LLM** 按 **token** 計費，呼叫次數沒有自然上限
- 三、空帳號實作分三層：必做只有你一個人，四步走完就及格

# agent 要有自己的 Role

agent 是一支程式，程式走 Role；權限只給它的任務需要的



- 1 綠色模型在 policy 的 Resource 有列；紅色沒列，所以拒絕
- 2 底下紅框：把你的 access key 貼進設定檔，agent 就變成你
- 3 agent 有自己的 Role，使用者的授權是另外一層

# 權限給多寬，出事範圍就多寬

| 做法                    | 出事時                             |
|-----------------------|---------------------------------|
| 貼你的 <b>access key</b> | agent 就是你，它刪 bucket 等於你刪的，追不到是誰 |
| 掛管理者 <b>Role</b>      | 它有多大權限，就能做多大的錯事                 |
| 掛專用 <b>Role</b> ，寫具體  | 它只能做那幾件事，出事範圍就那幾件               |

三列對回三條規則：長期金鑰不落地、最小權限、沒寫就拒絕。

# 模型呼叫權限可以細到指定模型

白話：只准在東京叫 Claude 系列，其他都不行

```
agent-role policy
{
  "Effect": "Allow",
  "Action": [
    "bedrock:InvokeModel",
    "bedrock:InvokeModelWithResponseStream"
  ],
  "Resource":
    "arn:aws:bedrock:ap-northeast-1::foundation-model/anthropic.claude-*"
}
```

1 成本：不讓 agent 自己跳去用最貴的模型

2 合規：不讓它用公司沒核准的模型

# 忘了關是漏水，agent 跑歪是水管爆掉

傳統雲端按時間計費；LLM 按輸入 token 加輸出 token 計費，各自單價

傳統雲端：按時間

一台 EC2 一個月

時薪固定，一個月最多 730 小時  
上限算得出來，忘了關也就那個數

LLM 呼叫：按 token

每次呼叫都是新帳

輸入 token 加輸出 token 各自計價  
呼叫次數沒有上限，迴圈跑歪就一直燒

守法

Budgets 一個月一個數  
忘了關頂多虧一個月

守法

Budgets 加 Budget Action 自動鎖  
Bedrock 的 token 配額、agent 的最大步數與逾時

同一個 Budgets，兩種帳的風險形狀不同：左邊是漏水，右邊是水管爆掉

- 1 左邊一個月最多 730 小時，上限算得出來
- 2 右邊每一輪都在花錢，呼叫次數沒有自然上限
- 3 錯誤的形狀：agent 卡迴圈、retry 沒設上限、prompt 越滾越長

# 每個 **agent** 都要有三個上限

三個都是程式碼裡的一行設定，比 Budgets 快；Budgets 加 Budget Action 是它們之後的第二道

## 1 最大步數

agent 最多想幾輪。

---

## 2 逾時

單次呼叫最久幾秒。

---

## 3 每次呼叫的最大輸出 token

每次呼叫最多吐幾個 token。

**token 是錢**：prompt 每多一段，都是每次呼叫都要付的固定成本。

“

**agent** 是員工不是分身，  
給它自己的識別證跟自己的預算。

—— 講師整理 ——

## QUIZ 06

**agent** 卡在迴圈，一小時內叫了模型三千次，哪一層最先擋得住？

A

Budgets 的 email 通知

B

Cost Explorer

C

程式碼裡的最大步數上限

D

月底帳單

想 30 秒，等一下翻下一頁對答案。

## QUIZ 06 · ANSWER

**agent** 卡在迴圈，一小時內叫了模型三千次，哪一層最先擋得住？

A Budgets 的 email 通知

B Cost Explorer

C 程式碼裡的最大步數上限

D 月底帳單

C。應用層上限是秒級生效；Budgets 的花費資料有延遲，是第二道；Cost Explorer 跟月底帳單都是事後。

# 動手做：算出 agent 迴圈跑歪一晚要花多少錢 (1/2)

| 小步 | 做什麼                                                                                                      | 做完應該看到                                                        |
|----|----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| 1  | 開 Amazon Bedrock pricing，找到 Anthropic models 的表格                                                         | 表頭有 Price per 1M input tokens 跟 Price per 1M output tokens 兩欄 |
| 2  | 挑一個 Claude 模型那一列，抄下輸入跟輸出兩個單價                                                                             | 兩個美元數字，輸出單價比輸入單價高                                             |
| 3  | 算正常的一次任務：5 步，每步輸入 2,000 token、輸出 500 token。公式是「步數 × (2,000 ÷ 1,000,000 × 輸入單價 + 500 ÷ 1,000,000 × 輸出單價)」 | 一個很小的金額，通常不到 1 美元                                             |

# 動手做：算出 agent 迴圈跑歪一晚要花多少錢 (2/2)

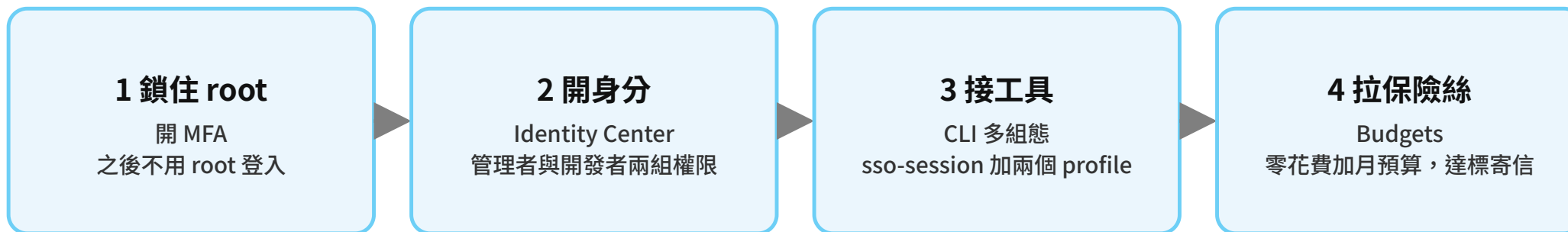
| 小步 | 做什麼                                    | 做完應該看到                                                                                                                                       |
|----|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| 4  | 同一個公式，步數改成 2,880（迴圈卡住，每 10 秒一步，跑 8 小時） | 在每步 token 固定的假設下，金額是上一步的 576 倍；如果每一步還把前面的對話都塞回 prompt，輸入 token 越滾越多，金額還會再往上                                                                  |
| 5  | 對照本課的應用層三個上限，寫一行：「最大步數設多少，第 4 步就不會發生」  | 一行答案，例如「最大步數 10；在本題每步 token 固定的假設下，最多花第 3 步的兩倍」。真的要封頂，本課三個上限都要，各擋一件事：最大步數擋輪數（retry 也算一輪）、逾時擋單次卡住、最大輸出 token 擋每輪回覆的長度；輸入越滾越長是第四件，要靠程式自己截斷歷史對話 |

這一步讓你親眼看到本課「LLM 的錢怎麼燒」那一段講的事：同一個單價，錯的是步數，沒有自然上限，所以要在程式裡先設最大步數，Budgets 是第二道。

# 空帳號四步，順序不能換

情境：一個全新的帳號，老闆說每個月不要超過 30 美元；零程式碼，全在主控台跟 CLI 上做

一個空帳號交給三人團隊，四步做完才算能用



## 驗收全部用指令，不看截圖

`aws sts get-caller-identity --profile dev` 回你的帳號  
`aws iam get-account-summary` 的 `AccountMFAEnabled` 是 1  
`aws budgets describe-budgets` 列出兩筆

- 1 root 沒鎖之前，不要開任何人的權限
- 2 身分沒開之前，CLI 沒東西可接
- 3 保險絲最後拉，但一定要拉；下排是驗收指令，每一步做完跑一條

# 前面的動手做，在這裡接起來

| 動手做                                         | 在 Lab 哪一步再用到                                                                           |
|---------------------------------------------|----------------------------------------------------------------------------------------|
| 看到東京有幾個 <b>AZ</b> ，台北 <b>Region</b> 預設沒開    | 2-1 右上角區域切到 Asia Pacific (Tokyo)                                                       |
| 把 <b>AmazonS3ReadOnlyAccess</b> 唸成一句中文      | 2-1、2-3 挑 AdministratorAccess、PowerUserAccess 兩份 AWS 管理的 policy；進階 7 替 mei 讀懂自訂 policy |
| 看一個 <b>Role</b> 身上的兩份 <b>policy</b> ，再找它的金鑰 | 1-2 看 root 的 Access keys；2-3 自己建一個 dev Role，3-2 扮演它拿到會過期的臨時憑證                          |
| 用 <b>Well-Architected Tool</b> 審第一題就刪掉      | —                                                                                      |
| 估一台東京的小機器一個月多少錢                             | 4-2 月預算金額 30 美元怎麼定                                                                     |
| 算出 <b>agent</b> 迴圈跑歪一晚要花多少錢                 | 4-1、4-2 預算通知，它是應用層上限之後的第二道；加分 9 的 action 只擋開新 EC2，擋不住模型呼叫                              |

一句話：前面只看不改的那些畫面，到這裡換你真的動手設定。

# 分三層交付，只交必做就及格

三個人一次到位太多；必做做完，後面每一個 AWS 實作就都有安全的起點

| 層  | 做什麼                            | 評分          |
|----|--------------------------------|-------------|
| 必做 | 只有你一個人：看方案、鎖 root、開身分、接工具、拉保險絲 | 70%，只交這層就及格 |
| 進階 | 把阿德跟小美加進來，三個人各有各的權限            | 20%         |
| 加分 | Budget Action：月預算到 100% 自動煞車   | 10%         |

後面單元的實作只需要你能用[管理者身分登入主控台](#)；這份卡住了，後面照樣做得下去。

# 必做一：root 開 MFA，不留 access key

| 小步  | 做什麼                                                                                                                     | 做完應該看到                      |
|-----|-------------------------------------------------------------------------------------------------------------------------|-----------------------------|
| 1-1 | 用 root 登入，右上角帳號名稱 → Security credentials → Assign MFA device，選 Authenticator app，用驗證器 App 掃 QR code，連續輸入兩組六位數，按 Add MFA | MFA 那一區出現一個裝置               |
| 1-2 | 同一頁往下看 Access keys                                                                                                      | 沒有任何 access key；有的話按 Delete |
| 1-3 | 到 IAM 首頁看 Security recommendations                                                                                      | Add MFA for root user 是綠勾   |

root 沒鎖，拿到 root 密碼的人就能改掉後面所有設定。

# 必做零：先看帳號方案，Free plan 走路線 B

Free plan：開新帳號可選的方案，六個月內或點數用完前不收費，先到者為準；課堂帳號多半是它

| 小步  | 做什麼                                                                                                                         | 做完應該看到                   |
|-----|-----------------------------------------------------------------------------------------------------------------------------|--------------------------|
| 0-1 | root 登入，開 Billing and Cost Management 首頁，找 Free Tier 或 Cost and Usage 區塊                                                    | 寫著 Free plan 或 Paid plan |
| 0-2 | Free plan：第 2、3 步走路線 B（IAM 使用者加 Role）；Paid plan 或公司帳號：走路線 A（Identity Center；公司帳號要是 Organizations 管理帳號，不是就請管理者做 A2-1 到 A2-3） | 記住自己的路線，後面每一步都有標         |

啟用 Identity Center 會建立 AWS Organizations；Free plan 一加入就立刻升級付費、免費點數當場失效。

## 必做二（路線 B）：course-admin 加 MFA，再建 dev Role

| 小步  | 做什麼                                                                           | 做完應該看到                                  |
|-----|-------------------------------------------------------------------------------|-----------------------------------------|
| 2-1 | IAM → Users → course-admin → Permissions                                      | 有 AdministratorAccess；沒有就補              |
| 2-2 | Security credentials → Assign MFA device，選 Authenticator app                  | MFA 一個裝置                                |
| 2-3 | Roles → Create role → AWS account → This account → 勾 PowerUserAccess → 名稱 dev | Roles 多一個 dev；Principal 是你帳號的 root ARN  |
| 2-4 | 登出 root；登入頁選 IAM user，填帳號 ID、course-admin、密碼、MFA                              | 右上角顯示 course-admin                      |
| 2-5 | 右上角帳號名稱 → Switch role，填帳號 ID 跟 dev                                            | 右上角變成 dev；IAM → Users 按 Create user 會被拒 |

沒開新金鑰；Switch role 就是扮演 Role，拿到會過期的臨時身分，跟 Identity Center 學的是同一件事。

# 必做二（路線 A）：用 Identity Center 開自己的身分

只給 Paid plan 或公司帳號；Free plan 的人不要按 Enable

| 小步   | 做什麼                                                                                                   | 做完應該看到                                          |
|------|-------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| A2-1 | 區域切到 Asia Pacific (Tokyo)，搜尋 IAM Identity Center → Enable，選預設的 organization instance                  | 左側出現 Users、Permission sets；記下 access portal URL |
| A2-2 | Users → Add user，填自己的 email，使用者名稱叫 me；收信設密碼，第一次登入 access portal 時照畫面註冊 MFA                            | Users 列表有 me                                    |
| A2-3 | Permission sets 建 AdministratorAccess、PowerUserAccess 各一個；AWS accounts → 勾帳號 → Assign users → me，兩個都勾 | 帳號底下顯示 me 有兩種權限                                 |
| A2-4 | 用 me 登入 access portal，點 PowerUserAccess 進主控台                                                          | IAM → Users 按 Create user 會被拒                   |

**PowerUserAccess**：AWS 內建的政策，除了 IAM 跟 Organizations 之外什麼都能做。

# 必做三：CLI 開一個 dev profile，兩條路線都拿臨時憑證

路線 B 沒開新金鑰：dev 是拿 course 那把金鑰去換一小時的臨時憑證

| 小步  | 路線 B                                                                                                                                                                       | 路線 A                                                                                                                                                                                     | 做完應該看到                                                                                       |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| 3-1 | <code>~/.aws/config</code> 加<br><code>[profile dev]:role_arn</code><br><code>= arn:aws:iam::帳號</code><br><code>ID:role/dev、</code><br><code>source_profile = course</code> | <code>aws configure sso:session</code> 填<br><code>course</code> ，貼 <code>access portal URL</code> ，<br>選 <code>AdministratorAccess</code> ， <code>profile</code><br>填 <code>admin</code> | B：檔案存好；A：瀏覽器授權頁按 <code>Allow</code>                                                          |
| 3-2 | <code>aws sts get-caller-</code><br><code>identity --profile dev</code>                                                                                                    | 再跑 <code>aws configure sso</code> 選<br><code>PowerUserAccess</code> ， <code>profile</code> 填<br><code>dev</code> ，再跑同一條指令                                                                | Arn 有 <code>assumed-role/dev (B)</code> 或<br><code>AWSReservedSSO_PowerUserAccess (A)</code> |

後面單元寫 `--profile admin` 的地方，**路線 B 一律用 `--profile course`**。

## 必做四：兩筆預算，月預算 30 美元

主控台要用管理者身分：路線 B 右上角 Switch back 切回 course-admin，路線 A 在 access portal 點 AdministratorAccess

| 小步  | 做什麼                                                                                                                   | 做完應該看到                             |
|-----|-----------------------------------------------------------------------------------------------------------------------|------------------------------------|
| 4-1 | 搜尋 Budgets → Create budget → Use a template → Zero spend budget，填你的 email                                             | 列表多一筆                              |
| 4-2 | 再建一筆 Monthly cost budget，金額 30 美元，email 同上；進 Edit alerts 把 85% 那條改成 80%，100% 保留                                       | 列表共兩筆；預設門檻以畫面為準（講師看到的是 85% 跟 100%） |
| 4-3 | <code>aws budgets describe-budgets --account-id 你的帳號ID --profile course</code> (路線 A 用 <code>--profile admin</code> ) | 回傳兩筆 Budget                        |

必做到這裡：交接文件要附 3-2 跟 4-3 兩條指令的實際輸出。

# 進階：加進阿德跟小美，先寬後收緊

你已經是管帳號跟帳單的小安；阿德做 EC2 跟 S3，小美只看 Cost Explorer 跟叫 Bedrock

| 步 | 做什麼                                                                                                                                     | 做完應該看到                                        |
|---|-----------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| 5 | 路線 B：User groups 建 devs 掛 PowerUserAccess，Users 建 ade 放進去、勾主控台登入、綁 MFA。路線 A：Identity Center 建 ade，指派 PowerUserAccess                    | ade 登入看得到 EC2，IAM 按 Create user 會被拒           |
| 6 | 把講師材料那份 policy 存起來：只給 bedrock:InvokeModel、ce:GetCostAndUsage 這類，Resource 寫到模型 ARN。B 存成 IAM policy finance-ai；A 做成 Custom permission set | Policies 或 permission set 列表多一個               |
| 7 | B：群組 finance-ai 掛那份 policy，建 mei（同樣勾主控台登入、綁 MFA）放進去。<br>A：把 permission set 指派給 mei                                                      | mei 開 Cost Explorer 看得到，按 Launch instance 會被拒 |

阿德先掛 PowerUserAccess，兩週後收緊；路線 B 的 ade、mei 都不要開 access key。

# 加分：Budget Action 自動煞車

執行後阿德還是能登入、能看、能停機器，只是開不了新的

| 步  | 做什麼                                                                                                                                   | 做完應該看到                          |
|----|---------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
| 8  | 建一個給 Budgets 穿的 Role：Trusted entity 選 AWS service → Budgets，權限照官方頁的表格給                                                                | Roles 列表多一個                     |
| 9  | 先把講師材料那份 Deny 建成 IAM policy；月預算 Edit → Add action：IAM role 選第 8 步的、門檻 100%、套那份 Deny、自動執行；對象 B 選 devs 群組，A 選阿德 permission set 對應的 Role | action 狀態是 Standby；不用真的花到 30 美元 |
| 解除 | 到 Budgets 把 action 狀態重設，或等月初自動重設                                                                                                      | 講師示範                            |

設定最多、最容易掛錯人；掛到小安身上，管理者自己會被鎖住。

# 提示卡：五個最常卡住的地方

| 小步     | 看到什麼                         | 怎麼辦                                                                       |
|--------|------------------------------|---------------------------------------------------------------------------|
| Enable | 說要建立 AWS Organizations 或升級方案 | 按取消；你是 Free plan，回第 0 步走路線 B                                              |
| 已升級    | 按下去了，帳號變 Paid plan           | 不會馬上扣錢，但免費點數沒了；立刻設好第 4 步兩筆預算，其餘照路線 A                                      |
| 2-5    | Switch role 說沒有權限            | 回 2-3 確認 Trusted entity 是 This account、course-admin 有 AdministratorAccess |
| 3-2    | Unable to locate credentials | ~/ .aws/credentials 要有第 1 單元建的 [course] 段                                 |
| 4-3    | AccessDenied                 | 用了 dev；Budgets 讀取要管理者，改回 course（路線 A 改 admin）                             |

# 個人作業：寫一頁交接文件

假設你明天離職，寫給接手這個帳號的人；個人完成

- 1 必做寫五項：Free plan 還是 Paid plan、root 的 MFA 裝置在哪、登入入口（B：帳號 ID 加 course-admin；A：access portal 網址）、你有哪兩種權限、兩筆預算的門檻
- 2 附上 3-2 跟 4-3 兩條指令的實際輸出，帳號 ID 遮掉
- 3 做了進階：加寫阿德跟小美各有什麼權限，附 mei 開 EC2 被擋的截圖或文字紀錄
- 4 做了加分：加寫 Budget Action 會做什麼，action 狀態是 Standby

**驗收** 必做 70%（五項齊全 40%、兩條輸出真的跑過 30%）、進階 20%、加分 10%；「root 的 MFA 裝置在哪」最常漏。

# 回家作業：六份，一律個人完成

各課的個人作業，交的東西都很短；實作的交接文件另外交，分層評分

## 選區

用四個問題替專題選一個 Region，  
每個問題一行理由。交五行。

## 讀 policy

挑一份官方 policy 範例，每條  
Statement 寫成一句中文。交三到  
五句。

## 身分圖

畫你本人、CI 流程、AWS 上的程  
式各用什麼身分。交一張圖加三  
行。

## 六根柱子

拿一個做過的選擇，六根柱子各寫  
一行加分、扣分或無關。交六行。

## 估一個月

用 Pricing Calculator 估專題一個  
月，再寫預算門檻。交連結加一  
行。

## agent 的 policy

寫一份 policy 加三個上限，各給數  
字跟理由。交 JSON 加三行。

## TAKEAWAY

- 01** Region 是你選的，AZ 是你分散的，Edge 是 AWS 替你放的。
- 02** 權限只寫在 Policy；安全的預設是根本沒有金鑰可以漏。
- 03** 六根柱子是替設計辯護的角度；帳單要月初就設好門檻。
- 04** agent 是員工，給它自己的識別證跟預算；帳號能不能交出去，看的是誰沒有權限。

# Reference

- **AWS Global Infrastructure ; Regions and Availability Zones** 三層與選區
- **IAM User Guide : Policies and permissions ; Security best practices** 讀 policy 與最小權限
- **IAM User Guide : IAM roles ; Temporary security credentials ; Enable MFA for root** 臨時憑證、鎖 root
- **IAM Identity Center User Guide ; CLI User Guide : Configure with Identity Center** 人怎麼登入
- **AWS Well-Architected Framework ; Shared Responsibility Model** 六大支柱與共同責任
- **Cost Management User Guide : AWS Budgets ; Budget actions ; Cost Explorer** 成本四工具
- **Billing User Guide : Using cost allocation tags** 標籤要先啟用
- **Bedrock User Guide : Identity and access management ; Bedrock pricing** agent 權限與 token 計費